NEXQON SCHOOL APP

Business document

Product scope, school operating workflows, access boundaries and pilot readiness

Version 1.1
13 October 2026

Implemented workflows, access boundaries and remaining decisions. Prepared from the application repository and recorded trial checks. No production service level or financial certification is implied.

Contents

1 Executive overview 3

2 Business purpose and expected outcomes 3

3 Users and access boundaries 4

4 School onboarding and account management 4

5 Teacher and parent workflows 5

6 School administration portal 6

7 Fee information and reporting 6

8 System ownership and information flow 8

9 Product scope and exclusions 8

10 Readiness and remaining work 9

11 Pilot acceptance scenarios 10

12 Decisions and commercial planning 10

13 Reference information 11

1 Executive overview

Nexqon School App gives schools a shared channel for homework, results, announcements and attendance, while giving parents a child-specific view of those updates. One role-aware mobile application serves teachers and parents. A separate web administration portal lets the platform owner onboard schools and lets school administrators manage their own school, users, classes, students, result uploads and fee information.

The current application is suitable for a controlled trial using synthetic data. Core workflows are implemented and connected to PostgreSQL services, with Kafka distributing business events. It is not yet a production-ready financial or student-data system: phone OTP is mocked, real device push and production attachment scanning are not configured, and operational controls require further validation. The Groq fee interpreter has been configured on the server and its live question-planning path was checked on 13 October 2026. That check returned an empty report, so it does not establish financial accuracy against a populated school ledger.

This document explains the product, operating workflows, access boundaries, business rules, trial acceptance criteria and remaining decisions. It does not approve prices, promise service levels or replace a school's privacy and accounting policies.

2 Business purpose and expected outcomes

Schools commonly distribute updates across separate messages, paper notices and spreadsheets. Parents may miss which child a message concerns; teachers may repeat the same update for each family; administrators may struggle to reconcile class membership and invitation records.

Nexqon centralizes these activities around a school, its classes and its guardian relationships. Expected benefits are clearer parent communication, less repeated publishing, traceable attendance follow-up and easier administration. These are intended outcomes, not measured savings or proven commercial results.

The initial audience is schools running a limited pilot, teachers assigned to those schools and parents linked to enrolled children. A suggested launch sequence is one school, a small number of classes and synthetic financial records, followed by a supervised pilot once production safeguards are enabled.

3 Users and access boundaries

Role	Main responsibilities	Access boundary
Platform owner	Create schools, provision school administrators, manage school status and review platform administration records	Platform administration does not automatically grant access to a school's private student results or fee reports
School administrator	Manage classes, teacher assignments, students and guardians; review attendance follow-up; publish bulk results; import and report fee information	Active school membership only; fee features require the School administrator role
Teacher	Publish homework, results and announcements; edit eligible posts; take attendance and follow up	Active teacher assignments and permitted class or subject audiences
Parent or guardian	Switch children, view updates and results, respond to absence notices and submit leave information	Children linked through active guardian relationships and the active school context

A person can hold multiple valid memberships. Switching roles or schools selects an existing authorized membership; it does not create new permissions. Teachers are not students, and a Parent membership represents a guardian, not a student login. An administrator who also has a Parent role may use that membership for their linked children, but the web portal remains an administration interface rather than a replacement for the parent mobile experience.

4 School onboarding and account management

Owner onboarding

The owner signs in, creates a school with its code, display name and timezone, and provisions the first school administrator. Subsequent administrator accounts are mapped to that school. School suspension and reactivation are administration controls; they must not silently expose another school's records.

School setup

The school administrator creates classes with an academic year and grade code, adds teachers and assigns them to one or more classes. Teacher class switching is available when multiple authorized classes exist and must be respected for publishing and attendance actions. Administrators can edit classes, teachers and students and manage their lifecycle.

Students have a school-specific admission number and a class enrollment. A guardian account is linked to the student. If the same parent phone number is used for another child, the existing account or membership is reused where appropriate; the second child is linked instead of creating a duplicate

active invitation. The parent can then select either authorized child in the mobile app.

Invitations and sign in

First activation uses the invited phone number and invitation code. Returning users can request OTP with their phone number. Identity resolves active roles and schools after sign in; a client cannot choose arbitrary school access. Sessions use expiring access tokens, rotating refresh tokens and logout.

The current trial uses development OTP 123456, not an SMS delivery provider. This makes access predictable and is unacceptable for production protection of real student or financial data. The SMS provider, verified-phone process, administrator protection and trial-to-production migration must be agreed before launch.

5 Teacher and parent workflows

Publishing and viewing school updates

A teacher selects an authorized class or audience, chooses homework, a result or an announcement, enters the details and publishes. Draft, upload, failure and retry states support interrupted work. Eligible content can be revised, archived or unpublished; revisions preserve history rather than overwriting published records without traceability. Scheduling and broader audiences depend on the permissions granted to that teacher.

Published updates are stored by Content and emitted as events. Read projections provide teacher post feeds and parent child timelines. A parent sees updates relevant to the selected child, not every post in the school. Results are student-scoped by default; broader publication or approval rules remain a school decision. Private attachment previews depend on permission checks and expiring file access.

Because feed projections are built asynchronously from Kafka events, publication and feed visibility are not one database transaction. A successful publish can precede the projected feed update. Delivery monitoring and replay procedures are necessary to distinguish short processing delays from failed event delivery.

Attendance and absence follow up

Teachers select an assigned class, record attendance and submit a versioned record. Absence events create parent-facing information and follow-up work. Guardians can acknowledge an absence, provide a reason and submit a leave note. Schools can review leave requests and escalation information.

Multiple guardians retain separate responses. The rule for treating an absence as acknowledged by any guardian or all required guardians is configurable and needs product confirmation. Attendance correction windows and treatment of late leave requests also need school policy approval.

Notifications and offline use

In-app notifications and deep links connect an update to its relevant screen. A persisted inbox entry does not prove delivery of a device push notification. Production FCM and APNs configuration, permissions and device delivery verification remain pending.

Native mobile builds include encrypted, role-scoped offline cache and an ordered mutation queue with conflict review. Cached data has a limited lifetime and is separated by role and school. Private results and attachments are not treated as unrestricted offline downloads. School administration and fee imports require online access.

6 School administration portal

The web portal organizes school administration separately from daily parent and teacher use. Administrators manage classes, teacher assignments and students, including multiple-class teacher access and guardian relationships. Logout and active-school context are part of the workflow.

Bulk roster processing supports school account setup. Excel result import parses the workbook in the browser, validates rows against the signed-in school's roster, provides a preview and publishes private student-scoped result posts. Invalid admission mappings must be corrected before publication. This is a result publishing workflow, not a full examination or grading engine.

Original workbooks are not retained as an unrestricted document repository. Validated import data is passed to the API, where school and role access are checked again. The administrator remains responsible for using the approved template, verifying preview rows and correcting the school's roster when admission numbers are missing or ambiguous.

7 Fee information and reporting

Excel import

The Fees area provides a template with Fees and Collections worksheets. Either worksheet can be supplied on its own. An administrator enters admission numbers, stable fee references, amounts, currencies and due dates; collections also require a stable receipt reference and collection date. Text admission numbers preserve leading zeros. Formula cells are rejected.

The current upload limit is 200 total rows and 5 MB. Admission numbers must resolve to one active student in the signed-in school. Unknown or ambiguous admissions, duplicate keys, invalid values, unsupported overpayments and inconsistent references block the batch. A preview identifies additions, updates, unchanged rows and errors before confirmation.

Confirmation commits all rows together. Stable invoice and receipt references let subsequent uploads update existing records without duplicating amounts. Retrying the same unchanged batch returns its original result. A changed ledger revision makes an old preview stale and requires review again; concurrent administrators do not silently overwrite a newer import. Omitted spreadsheet rows are not deleted.

Business definitions

Pending fees means the outstanding balance now on invoices whose due dates fall in the requested period, after subtracting all recorded receipts for those invoices. Without a period, it includes all recorded due dates, including future assessments. It is not a historical balance as of month-end and does not imply that every included invoice is overdue.

Collected fees means receipt amounts whose collection dates fall in the requested period. Upload dates and assessed invoice totals do not define collections. Reports show distinct student counts, record counts and exact totals separately for each currency; currencies are not converted or mixed into a single total.

Grade filtering uses current active class enrollments and configured grade codes for grades 1 to 12. Historical class membership and custom grade mappings are not implemented. School timezone defines current and previous month boundaries. These definitions should be confirmed with the school's finance team before operational reliance.

Voice and typed questions

An administrator can type a question or opt in to supported browser speech recognition, review the transcript and press Search. Voice never commits an import or performs a payment. Browser recognition can send audio to its speech provider; the fee interpreter receives text, not audio. A typed fallback remains available.

Examples include pending fees this month, collections last month and pending fees for grade three. The exact active school and interpreted filters are shown with the result. A question such as fees for the current month is ambiguous: the application asks whether pending balances or collections are intended instead of guessing.

Groq Free interpretation

The deterministic parser runs first and makes no AI request for supported wording. Otherwise, a configured Groq adapter can turn the question into a strictly defined report plan. The server supplies report definitions, the school's name and code, its timezone and current month boundaries. It does not send student lists, ledger rows, receipt amounts, database credentials or database execution tools. Administrators should still avoid personal details in questions because the question itself leaves the application.

The model does not write SQL or calculate financial totals. The gateway validates the plan, rejects invalid or extra fields and other-school selections, and resolves authorized grade membership. Fees validates the structured request again and runs existing parameterized database queries. Unsupported or unclear requests, malformed responses, provider errors and exhausted quotas produce a clarification without a report query.

Groq requires a protected API key and explicit confirmation that the account is on Free. The default model is `openai/gpt-oss-20b`, hosted by Groq rather than called through the OpenAI API. Conservative trial limits are three AI requests per minute, 100 per UTC day and one simultaneous AI request per gateway process. These application counters reset on process restart and are not a billing guarantee. Do not upgrade the provider account to a paid plan for this trial. Missing configuration retains the free parser.

This is grounded natural-language reporting, not a general RAG search engine, unlimited analytics or a promise to understand every possible question. Student-level filters, sections, multiple-grade comparisons, currencies, historical balances, refunds and forecasts are not currently supported. Automated provider tests use simulated responses. A separate live check on 13 October 2026 accepted an unfamiliar outstanding-balance question, requested clarification for an ambiguous question and

rejected another-school wording. No financial records were changed. The accepted query returned no matching records; schools still need populated-ledger acceptance tests, speech testing and quota monitoring.

8 System ownership and information flow

The mobile application uses React Native, Expo and TypeScript. The administration portal uses React, Vite and TypeScript. A NestJS and TypeScript API gateway coordinates internal services and presents role-shaped interfaces.

Component	Business responsibility
Identity	Invitations, authentication, sessions and membership selection
School	Schools, classes, students, guardian links and teacher assignments
Authorization	Configurable route and resource access decisions using live relationship facts
Content and Attendance	Published content revisions, attendance records and guardian workflows
Fees	Assessments, receipts, import history and authoritative fee reports
Files	Private file metadata, storage abstraction and controlled access
Notifications and Read Model	In-app inbox records and event-built feeds or summaries
Audit	Traceable administration and business events

Ten internal services operate behind the gateway. Nine own separate PostgreSQL databases; Authorization is stateless. Services reference external identifiers rather than using cross-database foreign keys. This is logical service and data isolation; the current free Render trial bundles services into one container rather than independently scaling each service.

A business service commits a change and its outbox event in the same database transaction. Its in-process publisher sends committed events to Kafka and retries failed delivery. Independent consumers update audit, notifications and read projections with deduplication. There is no separate worker service. Delivery is at least once, so consumers must tolerate repeated events. Fee report totals come directly from the authoritative ledger and do not wait for Kafka projections.

The trial web portal and API are hosted on Render, with remote Kafka connectivity configured separately. Hosting availability, cold starts, account quotas and database retention must be reviewed for the pilot. A free trial layout is not a production service-level commitment.

9 Product scope and exclusions

The implemented scope covers school onboarding, account and class management, teacher publishing and attendance, parent child timelines and responses, result imports and fee-information reporting. Fees are an approved extension to the original school communication scope.

The product does not currently process online payments, bank reconciliation, refunds, waivers, receipt cancellation or financial accounting. It also excludes transport tracking, chat or social feeds, an AI tutor, a complete examination engine and full school ERP operations. A natural-language fee assistant must not be presented as an accounting authority or a payment service.

10 Readiness and remaining work

Area	Current position	Required before production
Core school workflows	Implemented with service-backed access and automated regression coverage	Supervised school acceptance tests with approved policies and datasets
OTP authentication	Development code and invitation/session structure implemented	Real SMS delivery, abuse controls and verified account migration
Fee imports and reports	Implemented with exact amounts, atomic updates and concurrency checks	Confirm definitions, real template samples and operational finance review
Groq interpretation	Protected configuration and live planning checks completed; simulated provider tests also exist	Populated-ledger accuracy, browser speech, latency and quota checks; confirm provider account remains Free
Event delivery	Transactional outboxes, Kafka and independent consumers implemented	Deployment-specific lag, failure, replay and alerting checks
Attachments	Private storage abstraction and preview states implemented	Verified production bucket, scanning, expiry and lifecycle configuration
Device notifications	In-app inbox and deep-link workflows implemented	Real FCM and APNs setup and delivery testing
Distribution	Android trial build exists; one Android and iOS codebase	Store signing, distribution approvals and a verified iOS release build
Operations and compliance	Application controls and audit mechanisms exist	Backups, restore drills, retention, residency, support procedures and privacy review

Automated tests and local concurrent-read checks support engineering confidence but do not certify the free remote deployment for 100 simultaneous users. Production capacity requires realistic traffic tests, resource sizing and monitoring. Some role checks remain in gateway controllers as defense in depth; changing role permissions still requires review beyond editing policy configuration.

11 Pilot acceptance scenarios

1. The owner creates two schools. Each administrator sees only their active school and cannot import fees into the other school by changing request values.
2. A teacher assigned to two classes can switch classes for publishing and attendance. An unassigned class is rejected by the server, not merely hidden in the dropdown.
3. One parent phone number is linked to two students. The parent signs in once, switches children and sees the correct child's timeline, results and attendance information without duplicate activation records.
4. A teacher publishes and revises a post. Authorized parents receive the relevant view after projection processing; unauthorized families do not. Failures and processing delays are distinguishable.
5. A school uploads a fee workbook, corrects a receipt using its existing reference, then retries the upload. Totals remain correct, another school's admission number is rejected and a stale preview cannot overwrite a newer import.
6. Typed and voice-transcribed questions return the correct pending or collected report for a period and grade. Ambiguous questions ask for clarification. A provider outage leaves example questions usable without inventing a balance.
7. Logout and revoked memberships prevent subsequent protected access. Offline data remains scoped to the original role and school, and queued conflicts require review.

Recommended pilot measures are onboarding completion, successful sign ins, publish-to-visible delay, absence acknowledgement rate, import error rate, fee report accuracy against approved examples and failed provider requests. These are proposed measures; no achievement targets or results are asserted here.

12 Decisions and commercial planning

The application owner and schools should confirm result privacy and approval rules, guardian acknowledgement policy, attendance editing limits, fee definitions, retention and deletion, timezone defaults, supported languages and the production authentication provider. Data location, acceptable recovery time, backup frequency and support responsibilities require documented approval. Suggested recovery objectives are planning assumptions until restore tests establish them.

A school subscription or a time-limited pilot could be considered as a commercial model, but pricing, contracts, invoicing, subscriptions and billing enforcement are not implemented. Any proposed commercial launch should include hosting, messaging, storage, support and compliance costs rather than treating a free infrastructure trial as a sustainable cost model.

The current trial includes broader fee parsing and the configured Groq adapter. Production launch should follow verified authentication, private file handling, event monitoring, backups and school acceptance tests. Until then, participating users should treat the installation as a trial and use synthetic fee and student data.

13 Reference information

- Administration portal: https://schoolconnect-demo-admin.onrender.com/
- API gateway: https://schoolconnect-demo-api.onrender.com/api/v1
- Repository: https://github.com/bazato/schoolconnect-lite
- Groq structured outputs: https://console.groq.com/docs/structured-outputs
- Groq rate limits: https://console.groq.com/docs/rate-limits
- Groq billing FAQ: https://console.groq.com/docs/billing-faqs
- Browser speech recognition: https://developer.mozilla.org/en-US/docs/Web/API/SpeechRecognition

The supplied Kaksha design and product and technical specifications remain the baseline, supplemented by the approved school administration, service isolation, Kafka and fee-information changes.